

SINGAPORE-MIT ALLIANCE

Computational Engineering
CME5232: Cluster and Grid Computing Technologies for Scientific Computing
COMPUTATIONAL LAB No. 3
20th July, 2007

PARALLEL PROGRAMMING USING MPI

Objectives

To experiment MPI library functions focusing on point to point and collective communications.
To experience Condor – a High Throughput Computing System.

Background Knowledge

C/C++ languages and Basic MPI Functions

Exercises (Due date: 27th July 2007)

Please submit a short formal report recording your observations based on the assigned exercises and programming assignment in a week's time. The report should be complete and clearly written to show what you have done and what you have observed and interpreted from these exercises. All equations, symbols etc. if any must be explained clearly.

Matrix-Vector Multiplication

Consider a dense square matrix of order n and number of processes p , and assume that n is divisible by p . This matrix is stored in a row-major order in a file. You are required to develop a parallel program to carry out a matrix-vector multiplication as follows:

Question 1: Block-striped partitioning – column-wise block-striped partitioning

There are two types of block-striped partitioning, column-wise and row-wise block-striped partitioning. In this question, we are interested in the column-wise block-striped partitioning.

Write a function that reads in this dense square matrix. Process 0 should read in the number of rows n and broadcast n to all the other processes. Process 0 should then read in a block of n rows and distribute blocks of n/p columns to each of the processes such that the 1st n/p columns go to process 0, the next n/p columns go to process 1 and so on.

Once the distribution of the matrix is done, perform the parallel matrix-vector multiplication and print the result of the matrix-vector multiplication into an output file *result.txt*.

Question 2: Block-striped partitioning – row-wise block-striped partitioning

Repeat the procedures in (Q.1) for the matrix-vector multiplication using row-wise block-striped partitioning of the matrix. That means Process 0 should distribute blocks of n/p rows of the square matrix to each of the processes such that the 1st n/p rows go to process 0, the next n/p rows go to process 1 and so on.

Once the distribution of the matrix is done, perform the parallel matrix-vector multiplication and print the result of the matrix-vector multiplication into an output file *result.txt*.

Question 3: Block-checkerboard partitioning

Repeat the procedures in (Q.1) for the matrix-vector multiplication using block-checkerboard partitioning of the matrix. State the additional requirements for n and p in order to use block-checkerboard partitioning.

Question 4:

Comment on the differences between the column-wise block-striped partitioning in (Q.1), the row-wise block-striped partitioning in (Q.2) and the block-checkerboard partitioning in (Q.3) in the matrix-vector operation. (Specify the main points including the advantages and disadvantages of each approach)

Demonstrate all of the above by explaining what you have done, by writing a program, executing the program on the SMA Hydra cluster with the given sample file *matrix_vector_16.txt* of $n=16$ by using $p=4$, and showing the sample outputs.

Measure the timings required to compute the matrix-vector multiplication for $n = 16, 32, 64, 128, 256, 512, 1024$ with $p = 4, 16$. Comment on the timings (computational time and communication time).

Question 5: Embarrassing matrix-matrix multiplication

Choose one out of the three approaches for evaluating matrix-vector multiplication mentioned above. Write a new program that will compute a parallel matrix-matrix multiplication (*very* little modification from the above). For simplicity, the two matrices are square and have the same size n . Measure the timings required to compute the matrix-matrix multiplication for $n = 16, 32, 64, 128$ with $p = 4, 16$.